



Cloud PBX Integration for Contact-Center Queues & Agents

Delivered by: ASRP · **Role:** ASRP owned the PBX-integration workstream — design, implementation, and hardening · **Engagement type:** Independent contractor · **Domain:** Cloud telephony for freight logistics

Исполнитель: ASRP · **Роль:** ASRP полностью вела направление интеграции с АТС — проектирование, реализацию и доведение до боевой устойчивости · **Тип engagement:** независимый подрядчик · **Домен:** облачная телефония для грузовой логистики

1. Executive summary

ASRP designed and built the integration that lets a modern cloud communications platform operate on top of an **external cloud PBX** — RingLogix / Netsapiens¹. Organizations whose telephony already lives on that PBX keep using their existing PBX **queues (huntgroups), agents, and desk / mobile / soft-phone devices**, while the delivered platform becomes the single **system of record for the user interface**: agents log in and out of queues, pick a device, see who is on a call, and claim calls — all from one web screen, without switching to the PBX's own separate agent portal.

The core engineering challenge is that **two systems each own part of the truth**. The external PBX is authoritative for *live* telephony state (who is really in a queue, which device is registered right now); the platform database is authoritative for everything the UI reads. ASRP built the machinery that keeps those two views converged: **queue / agent-state management, device synchronization, call correlation** that ties a PBX call event back to the right platform call, and a **self-healing reconciliation layer** (inbound event webhooks plus a scheduled reaper) that repairs drift whenever a live event is missed.

This was the **largest project of the engagement** — a substantial subsystem that ASRP carried from an empty schema to a working, PBX-exercised system, and effectively a **single-owner subsystem** inside a larger team engagement (see §13).

2. Business context & the problem

Many contact centers run their phone system on a hosted cloud PBX and are not going to rip it out — the numbers, the carrier relationships, and the agents' desk phones all live there. But the PBX's own agent tooling is thin, and it is a separate world from the modern web platform where dispatchers, supervisors, and analytics actually live.

The objective was to let those organizations **keep their PBX telephony but drive it from the platform UI** — without forcing the UI to make a slow round-trip to the PBX on every screen refresh, and without the two systems silently diverging when an event is dropped. Concretely:

- **Agents** need to log in and out of a PBX queue and choose which device rings, from the web UI.
- **Supervisors** need a live, trustworthy view of who is available and who is on a call.
- **Inbound calls** routed by the PBX into a queue need to appear as first-class calls on the platform, so an agent can see and claim them with full context.
- The platform must **stay correct even when the PBX misses a notification** — no ghost logins, no stuck “on a call” state.

The hard requirement running through all of this: the screens must be **fast and always answerable**, and the mirror of PBX state must be **eventually correct even when the network isn’t cooperating**.

3. What ASRP was asked to do

The platform already had a call model, a web UI, and its own in-house queue routing. ASRP’s mandate was to bolt an entire *external-PBX* world onto it cleanly:

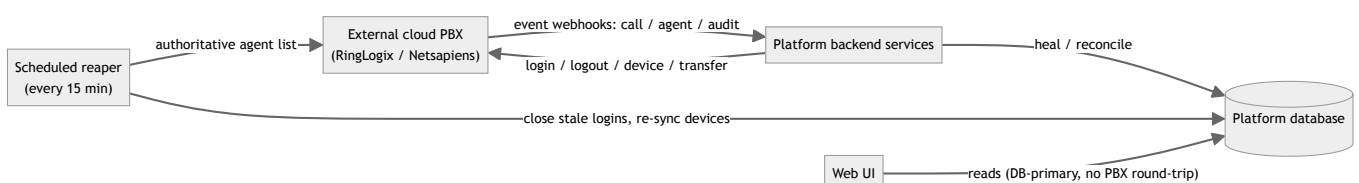
1. **Model the PBX domain** in the platform database — queues, agents’ devices, per-agent login state — as purpose-built tables plus a few columns on existing records.
2. **Let agents drive PBX queues from the UI** — queue login/logout with correct device selection, and provision an in-browser soft-phone.
3. **Make PBX-queue calls first-class** on the platform — correlate each PBX call event to the right platform call and auto-assign it to the answering agent.
4. **Keep the mirror honest** — a reconciliation layer that heals database state from PBX events and, as a backstop, a scheduled job that reconciles against the PBX’s authoritative view.

The four capabilities below (§4–§6) are what shipped; §7 tells the war-stories of what went wrong getting there.

4. System architecture

The design is deliberately **asymmetric about who owns the truth**, and that asymmetry is the key idea:

- The **external PBX** is authoritative for *live* external state — real queue membership and device registration.
- The **platform database** is authoritative for **UI reads**. Screens read from the database, never by calling the PBX on the hot path, so the UI stays fast and is always answerable.
- **Three convergence paths** keep the database mirror accurate: **direct mutations** (when the platform itself performs a login / logout / transfer), **inbound event webhooks** (the PBX pushes call / agent / audit events that heal the database in near-real-time), and a **scheduled reaper** (a periodic job that reconciles against the PBX’s authoritative agent list to catch anything missed).



Why a mirror + reaper + replay, instead of reading the PBX live?

The obvious naive design is: on every screen render, ask the PBX who's logged in and what devices are registered. ASRP rejected that, deliberately, for three reasons:

- **Latency and blast radius.** A supervisor dashboard refreshes constantly. Putting a third-party REST call on every read makes the UI as slow — and as *available* — as the PBX on its worst day. A dropped or throttled PBX response would blank the whole screen.
- **Rate limits and cost.** The PBX API is a shared, rate-limited resource. Fanning every UI read out to it does not scale to many concurrent supervisors.
- **The UI needs data the PBX doesn't hold.** Claim state, history, per-organization context, and correlation to platform call records only exist on the platform side.

So the platform keeps a **local mirror** it can read instantly, and treats the hard part — *keeping the mirror true* — as an explicit, multi-path convergence problem rather than pretending it away. Webhooks make the mirror fresh; the reaper makes it *eventually correct* even when a webhook is lost; direct mutations keep it correct on the happy path. No single path is a single point of failure. This is the whole architecture in one sentence: **read locally, converge continuously, and never let a reconciliation step destroy valid live state.**

5. A call in plain language (sample walkthrough)

Fictional but faithful to the real flow. *Dana* is a support agent at a freight brokerage whose phones run on the external PBX.

1. **Dana logs in.** She opens the platform, picks the “**Dispatch**” queue, and flips her status to available. The platform reads her devices, auto-selects her desk phone (she used it last), confirms with the PBX that the desk phone is genuinely a member of the Dispatch queue, tells the PBX to log her in, writes the login record, and re-reads the PBX to confirm both sides agree. Her status goes green on every supervisor's screen instantly.
2. **A call arrives.** A carrier dials the brokerage's number. The platform routes the call into the PBX Dispatch queue and — crucially — stamps the call record as a *PBX-queue call* and writes down the one identifier the PBX and the media gateway both agree on (see §6). The call appears in the queue on Dana's screen as claimable.
3. **Dana's phone rings and she answers.** The PBX rings the queue, her desk phone rings, she picks up. The PBX emits a call event (“this call is now active on extension 2XX”). The platform matches that event to Dana's on-screen call by the shared identifier, resolves *which agent* extension 2XX is, marks the call **Connected — Dana**, and stamps the answer time.
4. **Something drops.** Suppose the PBX never sends the “Dana logged out” event at end of shift. Fifteen minutes later the reaper walks the active logins, asks the PBX directly “is Dana still logged into Dispatch?”, hears “no”, and closes the stale login — no ghost agent left showing green.
5. **Soft-phone variant.** If Dana had chosen the in-browser phone instead of her desk phone, the platform would have provisioned a dedicated SIP device for her on the PBX — the rest of the flow is identical.

6. Call correlation (the hardest part)

The hardest part of the integration was keeping **phone state, queue state, and the supervisor's live view of both** in sync in real time: when the PBX routes an inbound call into a queue, that call has to show up as a real, claimable call on the platform, and every subsequent PBX event about it has to land on the **right** platform call and reach agent and supervisor screens correctly — with no ordering guarantee on when those signals arrive relative to each other.

The mechanism that makes this reliable is maintained in ASRP's engineering reference. Deeper technical detail is available under NDA.

7. Engineering war-stories

The interesting parts of this project were the failure modes that only show up against a live third-party system. Three worth telling — each as problem → diagnosis → fix.

7.1 The event that beat us to the wire (correlation race)

Problem. PBX-queue calls answer *very* fast. In production the PBX would emit its call event announcing that a queue call was already active **before** the platform had finished writing the correlation key for that call. The lookup in §6 would find nothing, and the call event — the one that assigns the call to the answering agent — was simply lost. Calls showed as unclaimed on-screen even though a human was already talking.

Diagnosis. This is a classic distributed race: two independent producers (the media gateway that gives us the correlation key, and the PBX that gives us the event to correlate) with no ordering guarantee between them. On slow calls the key won thanks to timing luck; on fast queue calls the event won. It was never going to be fixed by “just write faster.”

Fix — buffer and replay. When a PBX call event can't be correlated yet, it is **buffered in Redis under the call's identifier with a ~1-hour TTL** instead of being dropped. The moment the correlation key *does* get written (from the gateway webhook, or from the gateway-API fallback added precisely because queue calls answer too fast for the webhook), the platform **replays** the buffered events for that call. The replay path **drains the buffer atomically**, re-processes each event with **per-item error isolation**, and **re-queues** any item that still can't be matched (extending the TTL) rather than losing it. The net effect: early, late, and duplicated PBX events are all safe, and correlation becomes order-independent. This buffer-and-replay path was the single most-hardened piece of the integration.

7.2 The database that fell behind the PBX (reverse-heal)

Problem. The mirror was built on an implicit assumption that the platform is “ahead” of the PBX — it logs an agent in, then the PBX confirms. But drift happens in the *other* direction too: an agent's device re-registers and the PBX considers them logged in to a queue, while the platform database has **no login record** at all (a missed event, a restart, a manual change on the PBX side). The agent was live on the PBX but invisible on the platform.

Diagnosis. Treating the database as strictly downstream of platform-initiated actions was wrong. The PBX is authoritative for live state, so *the platform must be willing to learn state it didn't itself create*.

Fix — heal in both directions. When a device-registration event arrives, the platform checks the PBX’s authoritative agent list; if the PBX shows the agent logged in but the database has no login, it **creates the missing login record** (a “reverse heal”), and returns early so the general handler can’t immediately overwrite it. Healing now works symmetrically — webhooks can *create* a missing login as readily as they *close* an invalid one — and every decision is validated against the PBX’s authoritative list so a stale notification can’t manufacture a phantom login. A dedicated **active-call protection** rule was added in the same area: an agent who looks “unavailable” but is actually mid-call is never logged out; the call event resolves their state instead.

7.3 The claim that survived a failed transfer (rollback)

Problem. Claiming a PBX-queue call is a two-step act: mark the call claimed in the database, then tell the PBX to transfer the live call to the agent’s device. If the second step failed — the transfer returned unsuccessful, or the required routing data was missing — the *first* step had already committed. The result was a **silent false positive**: the platform showed the call claimed by an agent who never actually got connected.

Diagnosis. A local mutation and a remote side-effect were being treated as if they always succeed together. They don’t, and the local state must not outlive a failed remote step.

Fix — unclaim and throw. The claim path now treats the PBX transfer as the point of no return. If `transfer-to-device` returns unsuccessful **or** throws **or** the required data (domain / extension / correlation id) is missing, the platform **rolls back the local claim** (unclaims the call) and **raises a claim-failed error** so the UI reflects reality. The whole claim is serialized under a distributed lock so two agents can’t claim the same call concurrently. A false “claimed” state can no longer persist behind a failed transfer.

7.4 Built for multiple queues per organization

An organization can run more than one queue, and an agent can be logged into more than one of them at once — this was a ground-up design choice, not a retrofit. Queue-login state is keyed per **(organization, user, queue)** rather than per (organization, user), with the “only one active login” invariant enforced at that same granularity, so two different queue memberships for the same agent never collide. Device selection is scoped the same way: before letting an agent log into a specific queue with a specific device, the platform checks with the PBX that the device is genuinely a member of *that* queue’s huntgroup, so a device valid for one queue can’t silently be used to log into another.

One nuance ASRP is still refining: for organizations spanning **many** simultaneously active queues, there is a read/write source-of-truth scoping edge that gets more subtle as the queue count grows — an area of further refinement, called out here rather than papered over, not a shipped guarantee.

8. Technology stack

Layer	Technology
External cloud PBX (integration target)	RingLogix / Netsapiens² (REST API, OAuth2)
Backend	TypeScript on Node.js / Bun
Data access	Drizzle ORM (TypeScript, from a shared schema package)
Persistence	MySQL — PlanetScale serverless in production, mysql2 driver against MySQL 9 for local development, with a primary/replica split
Caching, locks, buffers	Redis — token cache, distributed locks (Redlock), correlation-event replay buffers
Realtime to UI	NATS pub/sub → WebSocket → frontend
Scheduling	Cron service (the reconciliation reaper)

“Which database did you use?” → **MySQL**, via the **Drizzle ORM** in TypeScript. Production runs on **PlanetScale serverless** (MySQL-compatible); local development runs the **mysql2** driver against **MySQL 9**. A primary/replica split is used, and queue-state reads that must be authoritative are pinned to the **primary**.

9. Data model

The integration is backed by a relational MySQL schema (Drizzle ORM). ASRP designed the following purpose-built tables (described conceptually — proprietary names and columns are generalized):

- **Device registry** — one row per PBX device per user: registration state (registered / unregistered), registration contact / IP / user-agent / expiry, the queue it belongs to, and last-seen / last-selected timestamps. Uniqueness enforced per (organization, user, device).
- **Device-registration history** — append-only registration snapshots for auditability, written only when the snapshot actually changes.
- **Current queue-login records** — one row per active login, with a **unique constraint guaranteeing at most one active login per (organization, user, queue)**, plus device, status, and login timestamp.
- **Queue-login history** — an archive of closed logins, so live-state tables stay small while history is preserved.
- **Queue (huntgroup) registry** — the set of queues per organization, linked to the phone-number configuration and an active flag.

ASRP also added a small number of **columns to existing platform tables**: a PBX-domain identifier on the organization, a PBX-extension identifier on the user, a per-number “this number is a PBX queue” flag, and — on the call record — the **indexed correlation key** and PBX-queue flag used by call correlation (§6), plus the answering-agent assignment and answer-time fields.

Data access is centralized in typed query helpers (create / close / replace login, sync devices, resolve organization-by-PBX-domain and user-by-PBX-extension), with authoritative queue-state reads pinned to the primary database.

10. Reliability & operations

- **Three-path convergence** — direct mutation, event webhooks, and the scheduled reaper together keep the mirror aligned with the PBX; no single path is a single point of failure.
- **Self-healing in both directions** — webhooks can *create* a missing login (reverse-heal, §7.2) as well as *close* an invalid one; the reaper independently reconciles against the PBX's authoritative agent list every **15 minutes**, single-flighted by a distributed Redis lock.
- **Active-call protection** — reconciliation never logs out an agent who is genuinely on a call, even if a transient signal suggests otherwise.
- **One-active-login invariant** — enforced at the database level (unique constraint) *and* by transactional login / replace helpers, so a race can't produce two active logins for the same queue.
- **Shared “effective login” definition** — a single predicate defines what “logged in” really means (accounting for available / manual / automatic states and whether a call is being delivered), used identically by the UI, the webhooks, and the reaper so they cannot drift apart.
- **Concurrency-safe token handling** — OAuth2 refresh guarded by an in-process promise *and* a distributed Redis lock, with proactive refresh and transparent one-shot 401 retry.
- **Out-of-order & duplicate event tolerance** — the correlation-event replay buffer makes early, late, or duplicated PBX events safe; failed items are isolated and re-queued rather than dropped (§7.1).
- **Failure-tolerant claims** — a failed PBX transfer rolls the local claim back rather than leaving a false “claimed” state (§7.3).
- **Realtime fan-out** — agent / queue changes publish over NATS to every relevant UI audience (per-user and per-group) so supervisor and agent screens stay live.

11. Maturity & status

Honest split of where the work stands.

Built & iterated (the four capabilities): agent queue login/logout with device auto-selection, device synchronization and soft-phone provisioning, call correlation with auto agent-assignment, and the webhook-plus-reaper self-healing layer. All are implemented and were exercised against the live external PBX.

Hardened (a dedicated mid-2026 engineering cycle): correlation replay-buffer robustness, distributed locking across the reaper and token refresh, huntgroup-scoping of device selection, simplification to DB-first reads on the hot path, and a transactional login model. ASRP also ran a self-initiated review-and-remediation pass over the subsystem as part of this cycle.

Forward-looking: the many-queues read/write scoping edge in §7.4 is an area of further refinement, not a shipped guarantee — it did not block the outcome below, since core multi-queue support (§7.4) is built and working, and the edge case only surfaces for organizations running many queues simultaneously.

Overall. The integration reached a working, PBX-exercised state and was piloted with a client (see §12 for what happened next). Development on this specific subsystem is currently **paused**: not because of a technical blocker, but because of a client-side build-vs-buy call made after the pilot (§12). The capability itself

remains a supported part of the platform for organizations that are already running their telephony on the external PBX. This page does not claim production-at-scale or general-availability status.

12. Results & outcome

The integration was built and exercised end to end against the live external PBX: agents logging into PBX queues from the web UI, inbound queue calls surfacing as claimable calls and auto-assigning to the answering agent, and the self-healing layer repairing state after missed events.

It reached a pilot stage with a client. At that point, the client's economics favored a different path: routing calls through the platform's own **in-house queue engine** was cheaper to operate than running production traffic through the external PBX. Because the agent-facing UI had already been built and proven end-to-end against the external-PBX integration, switching that organization over to the platform's internally-developed queue flow reused that same frontend investment rather than starting from scratch — the front end didn't need to be rebuilt, only re-pointed at the in-house routing engine.

This was a **client-side build-vs-buy decision made after piloting**, not a technical failure of the integration — the external-PBX capability worked as designed and remains available on the platform for organizations that are already committed to running their telephony on that external PBX. No adoption or performance figures are claimed beyond the pilot, because none have been measured on production-scale traffic.

13. What ASRP delivered

ASRP owned the **entire external-PBX integration workstream** on top of the existing platform: commit history on the subsystem runs roughly **95%+ ASRP, with minor contributions from a couple of other engineers**, i.e.:

- Designed the asymmetric **source-of-truth model** (PBX authoritative for live state, database authoritative for UI) and the **three-path convergence** that keeps them aligned.
- Built the full **agent queue login/logout** flow with huntgroup-aware **device auto-selection** and two-sided confirmation.
- Built **device synchronization** (registration state, queue assignment, history snapshots) and **soft-phone SIP provisioning** with encrypted credential storage.
- Solved **call correlation** via the stable SIP-Call-ID chain, including the Redis-backed **replay buffer** for out-of-order / duplicate PBX events.
- Built the **webhook-driven healing** layer (including reverse-heal and active-call protection) and the **scheduled stale-login reaper** (distributed-lock, single-flight).
- Implemented the **OAuth2 PBX client** with concurrency-safe token refresh and transparent 401 retry.
- Made **claims failure-tolerant** — rolling back a local claim on a failed PBX transfer.
- Delivered **realtime UI updates** over NATS / WebSocket for agent and queue state.
- Led the mid-2026 **hardening cycle** that took the integration from a working prototype through a client pilot.

14. FAQ (for client conversations)

Q: What database did you use? MySQL via the Drizzle ORM in TypeScript — PlanetScale serverless in production and the mysql2 driver against MySQL 9 locally, with a primary/replica split and authoritative queue-state reads pinned to the primary.

Q: Is this the same as your in-platform queue routing? No. The platform has its *own* in-house routing engine. This project integrates an **external** cloud PBX so organizations that already run their telephony there can drive their existing PBX queues and agents from the platform UI. They are two independent mechanisms, toggled per phone number.

Q: Two systems each hold state — how do you keep them from diverging? An asymmetric source-of-truth model plus three convergence paths: the platform performs direct mutations, the PBX pushes event webhooks that heal the database in near-real-time, and a scheduled reaper reconciles against the PBX's authoritative agent list every 15 minutes to catch anything missed. Self-healing works in both directions.

Q: How do you match a PBX call event to the right platform call? Via a stable-identifier chain: the external SIP Call-ID is the one id both the media gateway and the PBX agree on, so the platform writes it onto the call record as an indexed correlation key and matches PBX events against it. Events that arrive before the key is written are buffered in Redis and replayed once it's available.

Q: What happens when the PBX misses sending an event? The 15-minute reaper independently reconciles state against the PBX's authoritative agent list, closing stale logins and re-syncing devices. Webhooks and the reaper are complementary — neither is a single point of failure.

Q: Can it accidentally log someone out mid-call? No — reconciliation includes active-call protection and a shared “effective login” definition used consistently by the UI, webhooks, and the reaper, so a transient or stale signal can't close a valid on-call login.

Q: How is authentication to the PBX handled? OAuth2 with a Redis-cached token, proactive refresh before expiry, in-process dedup plus a distributed Redis lock so multiple instances never refresh simultaneously, and a transparent one-shot retry on token expiry.

Q: What was the hardest problem? Call correlation — specifically a race where fast-answering PBX queue calls emit their call event before the platform has written the correlation key. Solved with a buffer-and-replay design (§7.1).

Q: Why isn't this in active development anymore? It's paused, not abandoned: after a client pilot, the economics favored routing that client's calls through the platform's own in-house queue engine instead, and the front-end work built for this integration carried over directly. The external-PBX capability itself still works and remains available for organizations already on that external PBX.

Anonymized project description prepared by ASRP for portfolio use. Client name, proprietary source, internal identifiers, internal URLs, and security-operational details are intentionally omitted. Deeper technical detail is available under NDA.

1. **RingLogix / Netsapiens** — a commercial white-label cloud PBX / UCaaS platform (hosted call queues, huntgroups, SIP devices, and a REST API) that telecom service providers resell to their own business customers. Named here only as the third-party integration target; it is a public product.
<https://www.netsapiens.com>↩

2. **RingLogix / Netsapiens** — a commercial white-label cloud PBX / UCaaS platform (hosted call queues, huntgroups, SIP devices, and a REST API) that telecom service providers resell to their own business customers. Named here only as the third-party integration target; it is a public product.
<https://www.netsapiens.com>↩